

WHO PEN Production-Grade Medical RAG

الدليل التقني الشامل — Pipeline

Technical Architecture Reference

2026-08-19

WHO PEN Production-Grade Medical RAG Pipeline

الدليل التقني الشامل: من أساسيات RAG إلى هندسة نظام طبي قابل للإنتاج

الغرض من هذا الدليل التقني الشامل هو تقديم نظرة عامة على هندسة نظام RAG الطبي القابل للإنتاج. يغطي الدليل الأساسيات النظرية لـ Retrieval-Augmented Generation (RAG)، بل سجل معماري وهندسي متكامل لرحلة بناء نظام Medical RAG لمعالجة دليل WHO PEN المكوّن من 644 صفحة. تبدأ من الصفر للمبتدئ، ثم تصعد تدريجياً إلى تصميم Parsing و Hierarchical Chunking و Hybrid Retrieval و Cross-Encoder Reranking و Corrective RAG و Evaluation و Production Hardening، مع توثيق الأخطاء الفعلية التي ظهرت أثناء التنفيذ وكيف تم تشخيصها وإصلاحها.

حالة المشروع عند كتابة هذا المرجع: ingestion, parsing, table preservation, hierarchical chunking, dense + sparse indexing, تم إنجاز hybrid retrieval, reranking, parent expansion, corrective retrieval, grounded generation, evidence-pack evaluation, citation/numeric safety logic, Gold-100 retrieval evaluation, RAGAS answer-level evaluation معلقة مؤقتاً بسبب قيود خارجية في HTTP LLM judge endpoint (429)، لذلك لا يجب اعتبار النظام قد اجتاز Final Production Acceptance بعد.

فهرس المحتويات

مقدمة تأسيسية من الصفر

Document Pre-processing & Parsing Deep Dive

Layout-Aware Hierarchical Tree Chunking & Metadata

Advanced Retrieval Pipeline

Self-Reflective / Corrective RAG

Comprehensive Evaluation & Metrics

Troubleshooting & Engineering Trade-offs Log

Interview Questions & Model Answers

Production Tips & Future Roadmap

ملاحق هندسية

1. مقدمة تأسيسية من الصفر — Foundational Concepts for Beginners

1.1 ما هو الـ RAG؟

RAG = Retrieval-Augmented Generation، أي "التوليد المعزز بالاسترجاع".

الفكرة الأساسية بسيطة: بدلاً من أن نسأل الـ LLM سؤالاً ونطلب منه الاعتماد على ما تعلمه أثناء التدريب فقط، نقوم أولاً بالبحث داخل مصدر موثوق، ثم نعطي النموذج المقاطع ذات الصلة، ثم نلزمه بالإجابة منها.

Input → Processing → Output

Input - سؤال المستخدم، مثل: What is the recommended action for suspected acute stroke؟ - مستند طبي موثوق: WHO PEN.

Processing 1 تحويل السؤال إلى تمثيل بحثي. 2. البحث داخل فهرس المستند. 3. اختيار أكثر المقاطع صلة. 4. توسيع السياق إلى Parent Chunks عند الحاجة. 5. إعادة ترتيب النتائج باستخدام Cross-Encoder. 6. تمرير الأدلة إلى الـ LLM داخل Prompt مقيد. 7. فحص الاستشهادات والأرقام والادعاءات.

Output - إجابة Citation - grounded. - يشير إلى Refusal - Module / Section / Page أو Review Gate إذا لم توجد أدلة كافية.

المخطط النصي

```

User Question
|
v
Query Planning / Routing
|
v
Dense Search + BM25
|
v
RRF Fusion
|
v
Cross-Encoder Reranker
|
v
Parent / Table Expansion
|
v
Grounded Prompt
|
v
LLM Answer
|
+--> Citation Check
+--> Numeric Safety Check
+--> Unsupported Claim Check
|
v
Final Clinical Evidence Answer

```

WHO PEN من Use Case

في سؤال عن **AUDIT alcohol screening**، النموذج لا ينبغي أن "يتذكر" نطاق score من الذاكرة. الصحيح أن النظام يسترجع الصف أو البروتوكول الذي يحتوي threshold المطلوب، ثم يجيب من الجدول نفسه. هذا الفرق بين "نموذج لغوي يعرف أشياء" وبين "نظام Evidence Retrieval".

1.2 لماذا لا تكفي الـ LLMs بمفردها في المجال الطبي؟

الـ LLM ممتازة في اللغة، لكنها ليست قاعدة بيانات طبية مضمونة. توجد عدة مخاطر:

Hallucination: قد تنتج معلومة تبدو منطقية وغير موجودة في المصدر.

Knowledge Staleness: قد يكون التدريب أقدم من المستند أو guideline المقصود.

Source Ambiguity: النموذج قد يخلط بين WHO guideline ومصدر آخر.

Numeric Risk: الأرقام والجرعات والـ thresholds لا تتحمل تقريباً لغوياً.

Lack of Provenance: بدون RAG لا توجد سلسلة واضحة من Claim → Evidence → Page.

Institutional Scope: المطلوب أحياناً ليس "أفضل ممارسة عالمية" بل "ماذا يقول هذا المستند تحديداً؟".

في نظام طبي جيد، الـ LLM ليست مصدر الحقيقة، بل طبقة صياغة **Reasoning/Generation** فوق **Evidence Layer**.

Wrong Mental Model:

LLM = Medical Database

Correct Mental Model:

Trusted Medical Source = Evidence

Retriever = Evidence Locator

LLM = Controlled Language & Reasoning Layer

Verifier = Safety Enforcement Layer

1.3 مبدأ Garbage In, Garbage Out في RAG

إذا كانت مرحلة parsing سيئة، فلن تنقذك أفضل embedding model أو أقوى LLM.

صيغة مبسطة:

Final Answer Quality

≈ Parsing Quality

× Chunk Quality

× Retrieval Quality

× Generation Grounding

× Verification Quality

إذا كان أي عامل قريباً من الصفر، الناتج النهائي يتدهور بشدة.

مثال

افترض أن جدولاً في WHO PEN يحتوي:

AUDIT score
Interpretation

7-0

...

15-8

...

19-16

...

20≤

...

إذا قام parser بتحويله إلى:

0 7 8 15 16 19 20 interpretation intervention alcohol

فأنت فقدت **row semantics**. Embedding قد يعرف أن النص يتعلق بالكحول، لكنه لا يعرف أي interpretation يخص أي score.

1.4 لماذا WHO PEN مستند صعب للـRAG؟

المستند ليس كتاباً خطياً بسيطاً. يحتوي على:

644 صفحة.

Modules متعددة بمستويات مختلفة.

Headers وFooters متكررة.

صفحات متعددة الأعمدة.

جداول سريرية ومصفوفات تقييم.

.AUDIT tables

.5R's smoking cessation frameworks و 5A's

Flowcharts و بروتوكولات.

PowerPoint-like embedded slides والصور.

Facilitator notes بجوار أو بعد الشرائح.

.forms و Checklists

Thresholds ووحدات وأرقام حساسة.

Annexes داخلية و Annex نهائي في نهاية الكتاب.

التحدي الحقيقي ليس "تحويل PDF إلى نص" بل إعادة بناء البنية الدلالية للمستند مع **provenance**.

2. مرحلة المعالجة المسبقة — Document Pre-processing & Parsing

Deep Dive

2.1 الهدف الحقيقي من Parsing

الهدف ليس استخراج أكبر كمية نص، بل إنتاج **Canonical Evidence Representation** يمكن الوثوق به لاحقاً.

Input → Processing → Output

Input - who_pen_full.pdf - 644 صفحة.

Processing - Source audit. - Layout parsing. - Reading order. - Tables. - Text normalization. - Provenance capture. - Content-role classification. - Module/page mapping.

Output - Canonical element ledger. - Structured table ledger. - Page/module metadata. - Parsed document cache. - QA report.

```
PDF
|
+--> Raw Audit (page count, hash, text presence)
|
+--> Layout Parser
|   +--> Text blocks
|   +--> Headings
|   +--> Lists
|   +--> Tables
|   +--> Pictures / slide-like regions
|
+--> Reading Order Repair
|
+--> Repeated Noise Filtering
|
+--> Safe Text Normalization
|
+--> Provenance Ledger
|
v
Canonical Document Representation
```

2.2 Source Governance قبل أي Parsing

قبل قراءة المحتوى، يجب تثبيت هوية المصدر:

.Document ID

.Title

.Publisher

.Edition

.Publication year

.Page count

.SHA-256 hash

.Source URL

.Ingestion timestamp

هذا يمنع سيناريو خطير: أن يتغير PDF بينما يستمر النظام في استخدام embeddings قديمة.

لماذا الـ SHA-256 مهم؟

لأن اسم الملف غير كافٍ. يمكن أن يكون لديك ملفان باسم واحد لكن بمحتوى مختلف.

```

PDF bytes
  |
  v
SHA-256
  |
  +--> same hash -> cache MAY be reusable
  |
  +--> different hash -> invalidate downstream artifacts

```

2.3 Headers & Footers Stripping

المشكلة

المستندات الطويلة تحتوي عبارات متكررة في كل صفحة مثل:

عنوان الكتاب.

اسم المنظمة.

رقم module.

copyright.

page labels.

لو تم تضمينها في كل chunk، تصبح جزءاً قوياً من embedding space.

Vector Embeddings الأثر الرياضي المبسط على

افترض أن embedding للـ chunk:

$$[_i = _i +]$$

حيث: - ($i_$) = المعنى الحقيقي الخاص بالصفحة. - ($header$) = متكرر. - ($header$) = تأثير النص المتكرر.إذا تكرر ($header$) في آلاف المقاطع، ترتفع التشابهات بين chunks بسبب الـ header لا بسبب المحتوى السريري.

Cosine similarity:

$$[(_i, _j) = \{ | _i | | _j | \}]$$

وجود component مشترك كبير ($header$) يرفع similarity بصورة مصطنعة.

Input → Processing → Output

.Input: Text blocks + page positions
Processing: اكتشاف النصوص المتكررة في top/bottom bands عبر الصفحات ثم حذفها بحذر.
.Output: Content cleaner without repeated layout noise

قاعدة هندسية

لا تحذف كل ما يظهر في أعلى الصفحة بشكل أعمى؛ بعض headings مهمة. المطلوب هو **frequency + position + text identity**

Cross-Column Bleed, Reading Order Correction 2.4

المشكلة

Basic parser قد يقرأ عمودين بهذا الشكل:

```
Column A line 1
Column B line 1
Column A line 2
Column B line 2
```

بينما الصحيح:

```
Column A line 1
Column A line 2
...
Column B line 1
Column B line 2
```

هذا يسمى **Cross-Column Bleed**.

لماذا هو مدمر؟

لأنه ينتج chunk يجمع جملتين غير مترابطتين، فيصعب على embedding والreranker فهم المعنى.

الحل

استخدام layout-aware parser يلتقط:

.Bounding boxes

.Block types

.Reading sequence

.Column geometry

.Heading relationships

Page Coordinates

```
|
+--> Detect blocks
|
+--> Group by columns
|
+--> Sort within column
|
+--> Reconstruct semantic order
```

v

Correct Reading Sequence

Use Case

في صفحة تشرح CVD risk chart، قد يكون شرح الرسم في عمود و steps في عمود آخر. دمجها line-by-line يصنع تعليمات سريرية غير صحيحة.

Structured Elements Table Extraction 2.5

لماذا لا نعامل الجدول كنص عادي؟

لأن الجدول يحتوي بنية:

Cell(row, column)

وليس مجرد كلمات.

الطريقة الصحيحة

لكل جدول نحتفظ بـ:

table_id

page

caption

original headers

safe internal headers

rows

Markdown representation

HTML representation عند الحاجة

provenance

module/section metadata

مشكلة **duplicate headers** التي ظهرت فعلياً

بعض الجداول تحتوي أعمدة لها نفس الاسم أو خلايا header فارغة/متكررة. عند تحويلها مباشرة إلى DataFrame قد يحدث:

.overwritten columns

.warning

.row data loss

الحل الهندسي كان فصل:

Human Header: RR (95% CI)

Internal Key: col_03

ثم تخزين الاثنين:

```
{
  "internal_column": "col_03",
  "original_header": "RR (95% CI)"
}
```

وبذلك نحافظ على schema للمعالجة وعلى العنوان الأصلي للاسترجاع والعرض.

Input → Processing → Output

.Input: table cells + layout coordinates
Processing: reconstruct row/column grid, resolve merged cells, protect duplicate headers, serialize
.Output: Markdown/structured table with row semantics

Use Case: AUDIT

السؤال:

What intervention corresponds to an AUDIT score of 16–19?

لو الجدول محفوظ بنيوياً، يمكن للـ BM25 أن يلتقط 16-19، وللـ embedding أن يفهم alcohol intervention، وللـ reranker أن يرى العلاقة الكاملة داخل الصف.

OCR, Embedded PPTX / Slide-like Regions 2.6

المشكلة

بعض الصفحات تحتوي شرائح أو صور مدمجة. النص قد لا يكون موجوداً كـ PDF text layer.

تصميم مثالي

```
Page
|
+--> Native text exists? ---- yes ---> parse normally
|
no
|
+--> Picture / slide region detected
|
+--> selective OCR
|
+--> attach OCR text to page/slide provenance
|
+--> connect with facilitator notes
```

نقطة مهمة من تنفيذ المشروع

في النسخة المستقرة تم تجنب **global OCR/picture inference** على كل الصفحات للحفاظ على RAM في Colab، مع الاعتماد على native text/layout + table structure كمسار رئيسي. هذا Trade-off مهم: OCR الشامل يزيد التغطية النظرية لكنه قد يرفع الزمن والذاكرة ويخلق OCR noise. الاستراتيجية الأفضل للإنتاج هي **Selective OCR** للصفحات التي تثبت QA أنها تحتاجه.

أدوات محتملة

Docling: layout + table-aware parsing وكان المسار الرئيسي.

PyMuPDF: raw audit, page rendering, quick inspection, text sanity checks

LlamaParse: بديل بحثناه كخدمة parsing متقدمة، مناسب حين تقبل external parsing dependency/cost

2.7 Text & Symbol Normalization

في الطب، normalization يجب أن يكون محافظاً.

أمثلة آمنة

إزالة soft hyphen.

إزالة control characters.

دمج hyphenation الواضحة عند انقسام كلمة بين سطرين.

توحيد whitespace.

أمثلة خطرة

لا تقم آلياً بتغيير:

≥ إلى >.

≤ إلى <.

mg إلى g.

0.5 إلى 5. ثم فقد leading zero.

10-5 إلى 510.

Use Case

في stroke أو diabetes protocol، فرق رمز واحد قد يغيّر القرار السريري.

2.8 لماذا فشلت Basic

¶Parsers

المشكلة

Basic text extractor

Layout-aware approach

Two-column pages

يخلط الأعمدة

يحلل geometry

Tables

يحولها إلى tokens خطية

reconstructs grid

Headings

قد تُفقد

يحفظ item type

Lists

يفقد nesting

يحتفظ بالهيكل

Pictures/slides

غالباً لا نص

detectable regions + optional OCR

Provenance

page only أحياناً

page + block coordinates/type

Reading order

غير موثوق

layout-driven

Snapshot 2.9 من نتائج parsing الفعلية

خلال التنفيذ تم الوصول إلى parsing كامل للـ 644 صفحة مع مئات العناصر الجدولية والصورية. أحد التشغيلات الموثقة أظهر:

.pages 644

.Docling summary text items 7,234 في مرحلة

.tables 145

.pictures 953

.blank pages 633 pages with provenance؛ الصفحات غير الموجودة في provenance تبين أنها

canonical items without provenance 0 في QA النهائي لذلك المسار.

canonical elements 7,180 في إحدى مراحل البناء.

وفي final run manifest الأحدث:

parents = 1168

leaves = 1499

tables = 145

.device = CUDA

الاختلاف بين أعداد leaves عبر التشغيلات طبيعي لأن hierarchy أعيد بناؤه بعد إصلاح metadata والجداول.

3. معمارية التقطيع وتصميم الشجرة — Layout-Aware Hierarchical Tree

Chunking & Metadata

3.1 لماذا Chunking أصلاً؟

لا يمكن عادة embedding أو إرسال 644 صفحة في كل query. نحتاج تقسيم الوثيقة إلى وحدات يمكن:

البحث داخلها.

ترتيبها.

ربطها بسياق أكبر.

الاستشهاد بها.

3.2 Fixed-size Chunking

الفكرة

Every 500 tokens -> new chunk

المزايا

بسيط.

سريع.

.deterministic

لماذا يفشل في الطب؟

يمكن أن يقطع:

If BP is ...
[CHUNK BOUNDARY]
... refer immediately when ...

أو يفصل row من table عن header.

Input → Processing → Output

.Input:

.Processing:

linear

token

text

counter

.Output: equal-sized chunks

الحكم

مفيد للـ baseline, غير كافٍ كمقاربة نهائية لمستند سريري معقد.

3.3 Semantic Chunking

يقسم النص عند تغير المعنى بدلاً من عدد tokens فقط.

```

Sentence embeddings
|
Similarity between adjacent segments
|
Large semantic drop
|
Create boundary

```

الميزة

أفضل من fixed-size في النص النثري.

القصور

ما زال غالباً خطياً؛ لا يمثل hierarchy الطبيعية: Theme → Module → Section → Table.

3.4 Layout-Aware Hierarchical Tree Chunking

هذه كانت الاستراتيجية المختارة.

الشجرة

```

L1 Theme
|
+-- L2 Module
|
+-- L3 Parent Chunk (~500–760 tokens)
|
+-- L4 Leaf (~100–220 tokens)
+-- L4 Leaf
+-- L4 Leaf

```

في config النهائي:

.Parent max: 760 tokens

.Parent overlap: 12%

.Leaf max: 220 tokens

.Leaf overlap: 12%

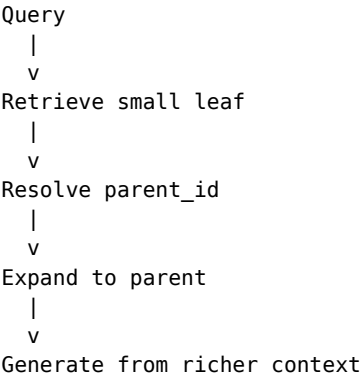
لماذا نعمل embedding للLeaves فقط؟

لأن leaf صغيرة ومركزة، فتكون retrieval precision أفضل.

لماذا نحتفظ بالParent؟

لأن answer generation يحتاج سياقاً أكبر.

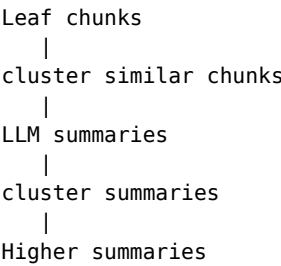
وهذا هو **Small-to-Big Retrieval**



Hierarchical Tree **RAPTOR 3.5** مقارنة بال

RAPTOR يعتمد على recursive clustering + summarization لإنشاء مستويات ملخصة فوق المقاطع.

تصور مبسط



مقارنة

المحور

Layout Hierarchy
RAPTOR

يعتمد على بنية المصدر

نعم

أقل

LLM summarization يحتاج

لا بالضرورة

نعم

تكلفة ingestion

أقل

أعلى

summary distortion خطر

منخفض

أعلى

مناسب للجداول والصفحات

قوي

يحتاج حماية إضافية

global conceptual questions

جيد

قوي جداً

citation إلى النص الأصلي

مباشر

يحتاج mapping دقيق

القرار للمشروع

WHO PEN لديه hierarchy حقيقية واضحة، لذلك Layout-Aware Tree كان أكثر قابلية للتدقيق. RAPTOR يمكن إضافته مستقبلاً كـ **secondary abstract retrieval layer** وليس بديلاً عن evidence tree الأصلية.

3.6 تصميم Metadata Schema

Metadata ليست زينة؛ هي جزء من retrieval correctness.

نموذج عملي

```
{
  "document_id": "who_pen_2018_training_modules",
  "theme_id": "theme_3",
  "theme_title": "...",
  "module_id": "3.4",
  "module_title": "...",
  "section_title": "...",
  "content_role": "background",
  "page_start": 312,
  "page_end": 312,
  "parent_id": "parent_...",
  "leaf_id": "leaf_...",
  "table_id": null,
  "token_count": 187,
  "source_sha256": "..."
}
```

وظيفة كل حقل

الحقل

الدور

document_id
source identity
module_id
routing/filtering
section_title

user-facing citation + relevance

content_role

...background/table/recommendation/activity

page_start/end

exact citation

parent_id

small-to-big expansion

leaf_id

retrieval unit

table_id

table lineage

source hash

cache validity

3.7 مشكلة Annex Metadata التي ظهرت فعلياً

أحد أكبر الأخطاء لم يكن في embedding، بل في **page → module mapping**.

الكود القديم التقط أول ظهور لعبارة Annex 1 وكأنه بداية Annex النهائي، ما أدى إلى تلويث metadata لمئات الصفحات.

Root Cause

```
Find("Annex 1")
|
+--> first textual mention
|
+--> WRONG assumption: final annex starts here
```

الحل

تحديد Annex النهائي الصحيح عند PDF page 629.

بناء module ranges غير متداخلة.

إضافة regression checks.

إعادة بناء hierarchy + embeddings + BM25 + Qdrant بعد التصحيح.

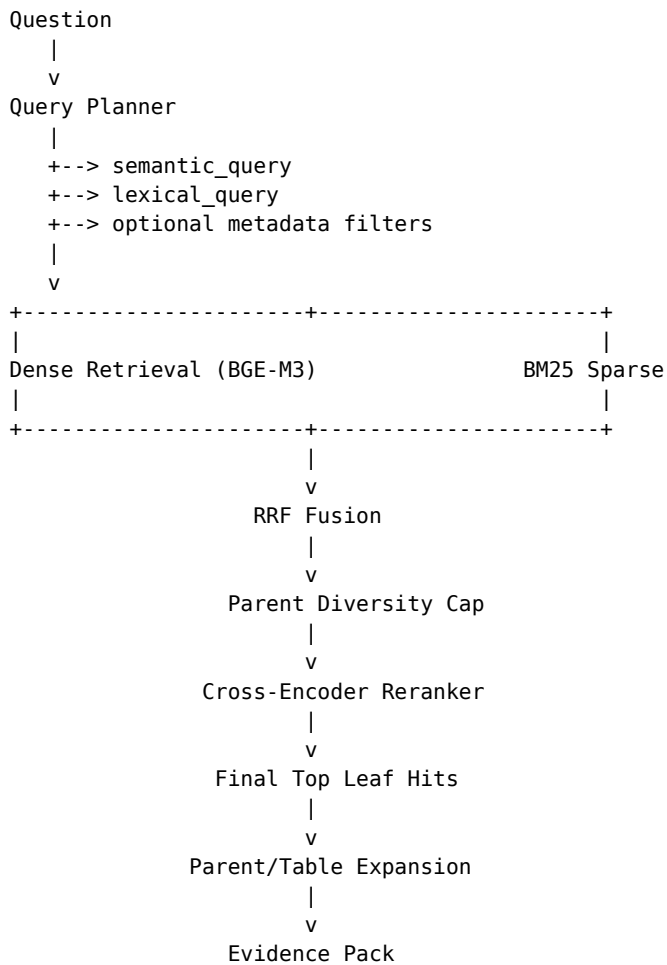
عدم إعادة Docling parsing لأن parsing نفسه لم يكن سبب الخطأ.

الدرس

Metadata corruption = retrieval corruption حتى لو vectors صحيحة.

4. محرك البحث والاسترجاع المتقدم — Advanced Retrieval Pipeline

4.1 الصورة الكاملة



4.2 Intent & Context Routing

الفكرة

بدلاً من البحث في 1,499 leaf لكل سؤال بنفس الطريقة، يمكن للـ **planner** تحديد **module** محتمل.

مثال:

Question: How is suspected acute stroke managed?

Planner:

```

semantic_query = "suspected acute stroke management"
module_id = "3.4" (only if highly confident)
  
```

Input → Processing → Output

.Input:		user		query
.Processing:	LLM	planner	/	deterministic fallback
.Output:	QueryPlan			

المستخدم QueryPlan

semantic_query
lexical_query
theme_id optional
module_id optional
content_roles optional
filter_confidence
patient_specific
out_of_scope
notes

Safety rule

Hard filtering لا يحدث إلا عند confidence مرتفع (0.92). وعند planner failure يوجد global fallback.

مشكلة مهمة ظهرت

في Q036 أعطى planner:

```
content_roles = ["background"]
filter_confidence = 1.0
```

فتفعل hard filter على content_role، مع أن evidence السريري قد يظهر في table أو training activity. الدرس: **content role لا ينبغي أن يكون hard filter منفرداً بسهولة.**

4.3 Self-Querying & Metadata Filtering

Self-querying يعني استخراج constraints من السؤال.

مثال:

"In module 3.4, what should be done for suspected stroke?"

يمكن تحويله إلى:

```
{
  "semantic_query": "suspected stroke immediate management",
  "module_id": "3.4"
}
```

Fallback Design

Apply hard filter

```
|
+--> enough results? yes -> continue
|
no
|
v
```

Retry global retrieval without hard filter

هذا مهم لأن false-negative metadata filter أخطر من noise إضافي في medical RAG.

4.4 Dense Retrieval — BGE-M3

Bi-Encoder

يحوّل query وال document chunks إلى vectors منفصلة:

[$q = f_{\text{enc}}(\text{query})$]

[$d_i = f_{\text{enc}}(\text{chunk}_i)$]

ثم similarity:

[$s_i = (q, d_i)$]

لماذا مناسب؟

سريع بعد indexing.

.semantic matching

multilingual capability مفيدة مستقبلاً للأسئلة العربية.

المشروع

.Model: BAAI/bge-m3

.Leaf embeddings فقط.

.Vector DB: Qdrant

.dense_k = 60

4.5 Sparse Retrieval — BM25

BM25 ممتاز للكلمات الدقيقة والأرقام والأسماء والاختصارات.

صيغة مبسطة:

[$\text{BM25}(q, d) = \sum \text{IDF}(t)$]

لماذا نحتاجه مع embeddings؟

سؤال مثل:

AUDIT 16–19

الأرقام قد تكون أقوى في lexical retrieval من semantic embeddings.

المشروع استخدم:

.bm25_k = 60

RRF و Hybrid Search 4.6

نحتاج دمج BM25 + dense دون محاولة مقارنة score scales المختلفة.

Reciprocal Rank Fusion

$$[\text{RRF}(d) = \frac{1}{\sum_{r \in \text{rankers}} \frac{1}{r}}]$$

في المشروع:

$$\text{rrf_constant} = 60$$

$$\text{fusion_k} = 50$$

مثال

Document A: - Dense rank = 2 - BM25 rank = 8

$$[\text{RRF}(A) = +]$$

Document B: - Dense rank = 1 - BM25 absent

قد يفوز A لأن rankers مستقلين دعموه.

لماذا RRF؟

لأنه robust ولا يحتاج calibration بين cosine و BM25 score.

4.7 Parent Diversity Cap

إذا كانت أفضل 20 نتيجة كلها من Parent واحد، يضيع recall لبقية الأدلة.

لذلك تم استخدام:

$$\text{max_leaves_per_parent} = 3$$

قبل أو أثناء fusion selection.

Parent A: leaf1, leaf2, leaf3, leaf4, leaf5

|
v

Keep at most 3

4.8 Cross-Encoder Re-ranking

Bi-Encoder vs Cross-Encoder

Bi-Encoder

encode(query) separately
encode(doc) separately
similarity

سريع ومناسب للـ large candidate pool.

Cross-Encoder

```

[query ; document]
  |
  v
Transformer jointly attends
  |
  v
relevance logit

```

أبداً لكنه أدق في fine-grained ranking.

المشروع

BAAI/bge-reranker-v2-m3

rerank_k = 30

final_leaf_k = 10

نقطة Calibration حرجة

القيمة:

`sigmoid(reranker_logit)`

ليست **calibrated probability**. لا يجوز وصفها كـ "احتمال صحة 87%" بدون calibration مستقل.

4.9 Auto-Merging / Context Expansion

بعد اختيار leaves، يتم تجميعها حسب parent_id واختيار أفضل parents.

المشروع:

final_parent_k = 5

max_context_tokens = 6000

لماذا؟

Leaf قد تحتوي نصف الفكرة. Parent يحتوي section أوسع.

Use Case: suspected stroke

Leaf retrieval أخفق في exact gold anchor، لكن Parent page 312 احتوى:

.suspicion criteria

.FAS

.time of onset

.activate emergency services

.immediate transfer

وهذا يوضح لماذا **leaf-level evaluation** وحدها لا تصف generation context بالكامل.

4.10 تقنيات إضافية تمت مقارنتها

HyDE

يولد LLM إجابة/وثيقة افتراضية ثم يعمل embedding لها.

مفيد عندما query قصيرة أو abstract، لكنه خطر في الطب لأن hypothetical text قد يضيف medical facts غير موجودة، حتى لو استخدم فقط للاسترجاع.

Multi-Query Retrieval

```
Original query
|
LLM generates 3-5 paraphrases
|
retrieve each
|
fuse results
```

يحسن recall لكن يزيد latency/cost وقد يخلق query drift.

Step-Back Prompting

يحول سؤالاً محدداً إلى مفهوم أعم ثم يبحث في الاثنين. مفيد لأسئلة conceptual، أقل ضرورة في threshold tables.

GraphRAG

ممتاز عندما السؤال يعتمد على علاقات متعددة الكيانات والمسارات. WHO PEN أساسه guideline hierarchies وليس knowledge graph dense، لذلك GraphRAG لم يكن الخيار الأول.

5. Self-Reflective / — مرحلة التوليد الذاتي المصحح ومكافحة الهلوسة

Corrective RAG

5.1 CRAG مقابل RAG التقليدي

RAG التقليدي:

retrieve once -> generate once

:CRAG

```

retrieve
|
assess evidence
|
+--> sufficient -> generate
|
+--> weak -> rewrite query -> retrieve again

```

في المشروع تم تحديد حد أقصى لدورتين **corrective retrieval loops** لمنع infinite loops.

5.2 Document Grading Node

Input

.top reranked leaves
.evidence pack
.relevance signals

Processing

هل يوجد evidence؟

هل أعلى relevance signal فوق gate؟

هل السياق يبدو كافياً؟

Output

SUFFICIENT / WEAK

ملاحظة

reranker sigmoid score لم نعد نتعامل معه كـ calibrated probability؛ gate يعتبر heuristic until calibrated.

5.3 Query Rewriting Loop

عندما retrieval ضعيف، يتم إرسال:

.original query

.previous plan

.top weak hits

إلى LLM ليعيد:

.semantic query

.lexical query

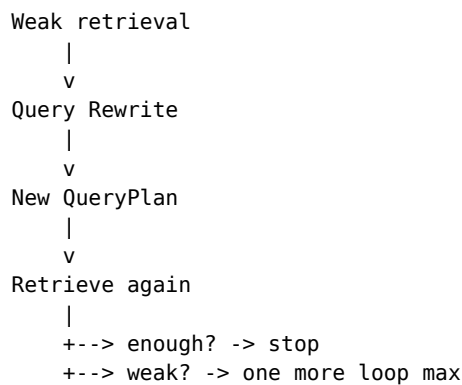
.optional safe routing

مع قواعد:

لا يخترع medical facts

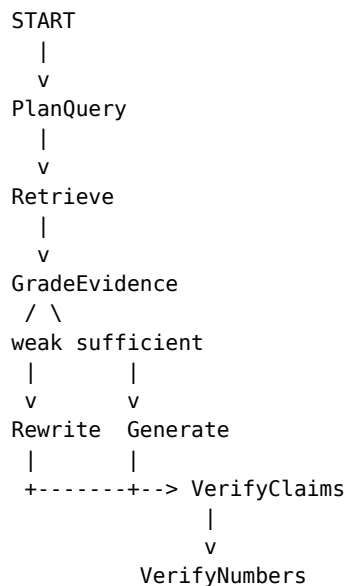
لا يفرض metadata filter إلا بثقة عالية.

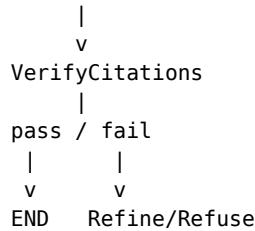
مخطط



5.4 LangGraph Mapping

المنطق الحالي يمكن تنفيذه functionally كما في notebook، أو تحويله إلى LangGraph في production.





لماذا LangGraph مفيد؟

- .explicit state
- .conditional edges
- .retry budget
- .observability
- .testable node boundaries

5.5 Grounded Generation with In-Text Citations

Prompt boundary

يجب أن يكون instruction واضحاً:

Use only the supplied WHO PEN evidence.
Do not introduce external clinical guidance.
Every material recommendation/threshold must cite evidence.
If evidence is insufficient, say so.

المقترح Output schema

```

{
  "answer": "...",
  "status": "supported",
  "citations": [
    {
      "evidence_id": "E1",
      "module_id": "3.4",
      "page": 312
    }
  ]
}

```

5.6 Hallucination Grader

بعد المسودة، نفكك answer إلى claims.

```

Answer
|
Claim segmentation
|
For each claim:
  retrieve supporting evidence id
  check entailment
|
unsupported claim?
yes -> remove/refine/refuse

```

في نظام طبي، الهدف ليس فقط fluent answer بل **claim-level support**.

5.7 Numeric Safety Guard

الأرقام الطبية تحتاج مساراً مستقلاً.

Input

.answer draft

.evidence texts

Processing

.extract numbers, ranges, units, inequality symbols

.map number to supporting evidence

.compare exact normalized representation

.block unsupported numeric claims

Example

Draft: refer if score ≥ 16

Evidence: 16–19 = brief counselling + continued monitoring

لا يكفي أن "16 موجود". يجب فحص **relation + range + recommendation**

5.8 Conditional Edges & Max Retry

:Never

while not good:
 rewrite forever

:Correct

max_retries = 2

ثم:

.Supported answer

.Insufficient evidence

.Human review

هذا يحمي latency والتكلفة ويمنع loops غير منتهية.

6. منظومة التقييم والتحليل المخبري — Comprehensive Evaluation & Metrics

6.1 لماذا لا يكفي Demo ناجح؟

سؤال واحد ناجح لا يثبت شيئاً تقريباً.

يجب فصل التقييم إلى:

- .Parsing QA
- .Retrieval evaluation
- .Evidence-pack evaluation
- .Answer-level evaluation
- .Citation/numeric safety
- .Failure analysis
- .Held-out production validation

6.2 بناء Gold Dataset

الحد الأدنى التعليمي

لـ MVP يمكن البدء بـ 15-20 سؤالاً موزعة على:

- .factual definitions
 - .numeric thresholds
 - .table lookups
 - .protocol steps
 - .contraindication/referral logic
 - .risk factors
 - .emergency guidance
- لكن هذا المشروع تطور إلى Gold-100 source-grounded dataset لتغطية 22 modules.

Gold item نموذجي

```
{
  "question_id": "Q056",
  "question": "How does WHO PEN characterize suspected acute stroke?",
  "module_id": "3.4",
  "relevant_leaf_ids": ["leaf_..."],
  "relevant_parent_ids": ["parent_..."],
  "reference_answer": "...",
  "answer_anchor": "Stroke is a medical emergency"
}
```

تحذير منهجي

بما أن Gold-100 استُخدمت في:

.label repair

.failure diagnosis

.ablation

.candidate selection

فهي أصبحت **development/tuning set** وليست pristine final test set. قبل claim إنتاجي قوي يجب بناء held-out جديد، مثلاً 30-50 سؤالاً، ويفضل clinician-reviewed.

6.3 Precision@K

[= Precision@K]

مثال

Top-5 فيها 3 relevant:

[$P@5 = 3/5 = 0.6$]

في الطب

Precision مهمة لتقليل noise الذي يصل للـ LLM.

6.4 Recall@K / Hit Rate

[= Recall@K]

إذا كان gold set غير exhaustive، نستخدم HitRate:

[$\text{Hit}@K = 1$ if any gold item appears in topK]

ثم المتوسط على الأسئلة.

لماذا **Hit@K** مهم؟

إذا evidence الصحيح غير موجود في generation Top-K، لا يستطيع تعويضه بأمان.

6.5 MRR

[$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}$]

يعاقب وجود الإجابة الصحيحة في rank متأخر.

6.6 nDCG@K

يقيس جودة ترتيب درجات relevance مع discount للمراتب المتأخرة.

مفيد عندما يوجد أكثر من relevant evidence بدرجات مختلفة.

6.7 النتائج الفعلية لـ Gold-100

بعد إصلاح gold labels غير المكتملة في بعض الحالات، baseline الموثق:

Metric
Result
Hit@1
0.7400
Hit@3
0.9400
Hit@5
0.9700
Hit@10
0.9700
MRR
0.830333
nDCG@10
0.857029

هذه أرقام قوية للـ development set، لكنها ليست production certification.

6.8 Evidence-Pack Evaluation

لأن generator لا يرى leaves فقط، تم قياس parents الفعلية التي تصل إليه.

Metric
Result
ParentHit@1
0.7600
ParentHit@3
0.9200
ParentHit@5
0.9500
AnchorCoverage
0.9300

insight مهم

بعض exact anchors لم تظهر حرفياً، لكن evidence parent احتوى الإجراء السريري المطلوب. هذا يكشف الفرق بين:

Exact-anchor metric
vs
Semantically sufficient evidence

لذلك يجب الجمع بين automated metrics و manual adjudication.

6.9 RAGAS Metrics

Context Precision

هل contexts التي تم جلبها فعلاً مفيدة للإجابة؟

High precision = قليل من noise.

Context Recall

هل retrieved contexts تغطي المعلومات المطلوبة في reference answer؟

Faithfulness

هل claims في answer مدعومة بال retrieved context؟

Answer Relevance

هل answer فعلاً تجيب عن user question بدون خروج جانبي؟

Production gates المخططة

context_precision ≥ 0.80

context_recall ≥ 0.80

faithfulness ≥ 0.80

critical numeric mismatches = 0

invalid citations = 0

unsupported clinical claims = 0

الوضع الحالي

RAGAS لم تكتمل لأسباب خارجية:

dependency mismatch متعلق بـ langchain_community.chat_models.vertexai.

بعد إصلاح LLM judge endpoint، dependency أعاد HTTP 429 حتى مع single-question probe.

لذلك RAGAS = **Pending**، وليس Failed quality metric.

6.10 Failure Analysis Methodology

لا تقل "retrieval failed" فقط. شخّص المرحلة.

taxonomy

Query Failure
Embedding Failure
BM25 Failure
Fusion Failure
Parent-Cap Dropout
Rerank Input Cutoff
Reranker Demotion
Chunk Boundary Failure
Metadata Filter Failure
Gold Label Incompleteness
Evidence Evaluation Granularity Mismatch

أمثلة فعلية

.Q042/Q075/Q080: reranker demotion

.Q056/Q083: rerank input cutoff في التشخيص الأولي.

.Q008/Q036: candidate reached retrieval channels لكن سقط قبل final top-k.

.Q033/Q067/Q083: ظهر لاحقاً أن gold labels لم تكن exhaustive بما يكفي.

هذه النقطة جوهريّة: لا تصلح retriever قبل أن تتأكد أن gold نفسه صحيح.

V2 Ablation Study 6.11 ولماذا رفضنا

تم اختبار تغيير عالمي يجمع:

.focused lexical rewrite

.rerank 50 instead of 30

.hybrid final score

النتيجة:

Hit@1 ارتفع.

MRR ارتفع.

لكن Hit@3/5/10 انخفض.

في Full V2 medical retrieval, coverage أهم من rank cosmetics, لذلك تم رفض Full V2.

ثم ablation أظهر أن A_lexical_only حافظ على coverage مع تحسن MRR صغير جداً، لكن المكسب لم يكن كافياً لتبرير complexity جديدة.

القرار الهندسي المحافظ: **freeze baseline** بدلاً من overfitting.

7. سجل الأخطاء والدروس المستفادة — Troubleshooting & Engineering

Trade-offs Log

7.1 سجل المشاكل الفعلية (نسخة محسنة للطباعة)

7.1.1 Basic PDF extraction

العرض: نص مختلط/جداول تالفة

Root Cause: parser خطي

الحل: Docling layout-aware parsing

الدرس: Parsing quality أساس RAG

7.1.2 Cross-column bleed

العرض: جمل غير مترابطة

Root Cause: reading order خاطئ

الحل: layout geometry

الدرس: لا تعتمد على raw text order

7.1.3 Duplicate table headers

العرض: فقد/استبدال columns

Root Cause: DataFrame keys غير فريدة

الحل: internal col_N + original headers

الدرس: schema ≠ display header

7.1.4 Annex contamination

العرض: modules خاطئة

Root Cause: أول Annex 1 اعتبر النهاية

الحل: final annex = page 629 + non-overlap checks

الدرس: metadata bugs قد تتفوق خطورتها على model bugs

7.1.5 Stale indexes

العرض: نتائج لا تطابق hierarchy

Root Cause: reused cache بعد rebuild

الحل: fingerprint embeddings/BM25/Qdrant

الدرس: cache invalidation لازم يكون source-aware

7.1.6 Qdrant local lock

العرض: reopen failure

Root Cause: multiple local clients

الحل: close/reopen safely

الدرس: local vector DB له lifecycle

7.1.7 NumPy/runtime incompatibility

العرض: import/runtime crash

Root Cause: package version interaction

الحل: preflight + stable dependency set

الدرس: verify env قبل expensive parse

7.1.8 Colab RAM instability

العرض: parser/vision pressure

Root Cause: global OCR/picture inference

الحل: native text/layout + selective OCR strategy

الدرس: accuracy-cost trade-off

7.1.9 Reranker “probability” misunderstanding

العرض: overconfidence

العرض: calibrated غير **Root Cause:** sigmoid(logit)

الحل: rename/interpret as score only

الدرس: calibration جزء من safety

7.1.10 LLM base URL issue

العرض: API call failure

Root Cause: config not reading env correctly

الحل: bind CFG.llm_base_url to environment

الدرس: config plumbing critical

7.1.11 Query planner content-role filter

العرض: evidence potentially excluded

Root Cause: hard filter on role

الحل: restrict hard filtering / fallback

الدرس: metadata filters يجب أن تكون conservative

7.1.12 Full V2 regression

العرض: بعض ranks أفضل و coverage أسوأ

Root Cause: multiple simultaneous changes

الحل: ablation + coverage-first selection

الدرس: لا tune 3 variables مرة واحدة

7.1.13 Incomplete gold labels

العرض: false failures

Root Cause: only one exact leaf labeled

الحل: manual source adjudication

الدرس: evaluator can be wrong

7.1.14 RAGAS import error

العرض: ModuleNotFoundError

Root Cause: package compatibility

الحل: pin compatible dependency / update imports

الدرس: evaluation stack needs version lock

7.1.15 Groq 429

العرض: RAGAS cannot run

Root Cause: external rate/token quota

الحل: pacing/alternate judge/quota upgrade

الدرس: judge availability منفصل عن RAG quality

7.1.16 API key in manifest

العرض: secret exposed

Root Cause: asdict(CFG) serialized key

الحل: remove secret, rotate key, env only

الدرس: never serialize secrets

7.2 السرعة مقابل التكلفة مقابل الدقة السريرية

Latency

مصادر latency:

.query planner LLM

.dense search

.BM25

.reranking 30 candidates

.corrective loop

.generation

.verifier calls

Cost

.LLM query planning

.generation

.verification

.RAGAS judging

.amortized Embedding + BM25 offline cost غالباً

Clinical Accuracy

لتحسين الدقة:

.retrieve more candidates

.rerank more

.use parent expansion

.verify claims

لكن كل هذا يزيد latency/cost.

قاعدة القرار

Medical production priority:

Safety / Evidence Coverage

> Rank cosmetics

> Latency optimization

> Cost optimization

مع الحفاظ طبعاً على SLO معقول.

7.3 Trade-off: Global OCR vs Selective OCR

Global OCR	Selective OCR
------------	---------------

تغطية عالية للصور	
-------------------	--

أقل تكلفة	
-----------	--

مرتفع RAM/latency	
-------------------	--

scalable	
----------	--

أكثر OCR noise	
----------------	--

targeted QA	
-------------	--

مفيد لل scanned PDF	
---------------------	--

أفضل لل mixed native PDF	
--------------------------	--

WHO PEN native-heavy, لذلك selective path كان عملياً أفضل في Colab.

7.4 Trade-off: Larger Leaves vs Smaller Leaves

Small leaves - retrieval precision أعلى. - خطر فقد context.

Large leaves - context أكبر. - embeddings أقل تركيزاً.

الحل: Small-to-Big.

7.5 Trade-off: Hard Metadata Filters

Hard filters تقلل search space لكنها ترفع خطر false negatives.

في medical RAG, false negative قد يخفي evidence الصحيح. لذلك hard filter يجب أن يكون:

.high-confidence

.validated field

8. بنك أسئلة الإنترفيو المتقدمة — Interview Questions & Model

Answers

Q1. لماذا اخترت Hybrid Retrieval بدلاً من Vector Search فقط؟

إجابة نموذجية: Dense retrieval ممتاز في semantic similarity لكنه قد يضعف مع exact tokens مثل AUDIT BM25 16-19, acronyms, units, drug names. lexical matching قوي في RRF يعطي robustness لأننا لا نحتاج calibration بين cosine score و BM25 score. في WHO PEN كان هذا مهماً خصوصاً للجدول والthresholds.

Q2. ما الفرق بين Bi-Encoder و Cross-Encoder؟

إجابة نموذجية: Bi-Encoder يقوم ب encoding مستقل لل query وال documents، لذلك يسمح بفهرسة مسبقة وبحث سريع. Cross-Encoder يعالج query+document معاً ويتيح full attention بينهما، فيعطي ranking أدق لكنه أبطأ. لذلك أستخدم Bi-Encoder لاسترجاع candidate pool ثم Cross-Encoder لإعادة ترتيب Top-N.

Q3. لماذا لم تستخدم Fixed-size chunks فقط؟

لأن guideline medical documents تحتوي tables, protocols, headings و logical boundaries. Fixed chunk يمكن أن يفصل شرطاً عن الإجراء أو row عن header. استخدمت layout-aware hierarchy و Small-to-Big: leaves للاسترجاع و parents للتوليد.

Q4. ما فائدة Parent ID؟

يربط leaf الدقيقة بال context الأكبر. أسترجم leaf لأنها مركزة، ثم أوسع إلى parent حتى يحصل generator على context كافٍ بدون embedding كل parent الضخم.

Q5. كيف تمنع Hallucinations؟

layered defense: 1. trusted source. 2. retrieval. 3. evidence استخدام واحدة. لا توجد تقنية واحدة. استخدام evidence. 4. prompt forbidding external knowledge. 5. citations. 6. claim sufficiency gate. 7. numeric guard. 8. refusal/human review.

Q6. ما أخطر Bug واجهك؟

Metadata mapping bug: أول ظهور لـ Annex 1 اعتبر بداية annex النهائي، فلوّث module metadata. المشكلة مهمة لأنها أثبتت أن retrieval failures ليست دائماً embedding failures. أصلحنا module boundaries وأعدنا بناء downstream indexes دون إعادة parsing.

Q7. لماذا لا تعتبر sigmoid(reranker logit) probability؟

لأن sigmoid مجرد monotonic transform للـ logit. بدون calibration على labeled validation set لا يمكن تفسير $0.8 \leq \text{probability} \leq 0.80$. يمكن استخدامه للـ ranking أو heuristic gate، لكن ليس confidence probability طبية.

Q8. كيف قيّمت retrieval؟

استخدمنا Gold-100 و Hit@1/3/5/10 و MRR و nDCG. بعد ذلك أضفنا evidence-pack metrics لأن generator يرى parents وليس leaves فقط. النتيجة كانت Hit@10=0.97 و ParentHit@5=0.95 و AnchorCoverage=0.93 على development set.

Q9. لماذا رفضت Candidate يحسن MRR؟

لأنه خفّض Hit@3/5/10. في medical RAG فقد evidence الصحيح أسوأ من رفع relevant document من rank 4 إلى rank 2 في بعض الحالات. استخدمنا coverage-first model selection.

Q10. كيف تفرق بين Retrieval Failure و Evaluation Failure؟

أعمل trace عبر anchor، same-parent match، reranker rank، fused rank، BM25 rank، dense rank، match، وأراجع source manually. وجدنا حالات كانت gold labels ناقصة، فلو عدلنا retriever بناء عليها لكننا overfit إلى evaluator خاطئ.

Q11. ما الذي يجعل Table RAG مختلفاً؟

الجدول يحتاج Flattening preserving row/column semantics. يحطم العلاقة بين value و header. لذلك أحتفظ بال retrieval text table- وأبني schema, original headers, row text, Markdown, table_id, provenance aware.

Q12. كيف تتعامل مع Query Routing؟

استخدم conservative planner. Routing optional, hard filter only when confidence high والفتر لا يستخدم value غير موجود في index metadata. إذا result count ضعيف نعيد global retrieval.

Q13. لماذا لم تستخدم GraphRAG؟

المشكلة الأساسية document hierarchy + protocols + tables أكثر من كونها multi-hop entity graph. GraphRAG يضيف ingestion complexity وتكلفة بدون دليل أنه يحل bottleneck الأساسي. يمكن إضافته لاحقاً إذا ظهرت أسئلة multi-hop cross-module حقيقية.

Q14. كيف تجهز النظام للإنتاج؟

أفضل offline ingestion عن online serving, أستخدم FastAPI, Qdrant persistent service, Redis cache, Docker, versioned indexes, health endpoints, structured logs, tracing, CI tests, security secrets, rollback strategy, manager, evaluation gates, canary deployment.

Q15. ما أهم درس معماري من المشروع؟

أقوى model لا يصلح data pipeline خاطئة. أكبر gains جاءت من تصحيح parsing/metadata/tables/evaluation labels, وليس من زيادة Production RAG model size. هو data systems + search + evaluation + safety وليس prompt فقط.

9. Production Tips & Future — نصائح للمستقبل وخريطة الإنتاج

Roadmap

9.1 فصل Online Pipelines و Offline

Offline

PDF -> Parse -> Canonicalize -> Chunk -> Embed -> Index -> Evaluate -> Version

Online

Request -> Route -> Retrieve -> Rerank -> Expand -> Generate -> Verify -> Respond

لا تتم بإعادة parsing داخل request path.

9.2 FastAPI Serving Architecture

```

Client
|
v
FastAPI
|
+--> /health
+--> /ask
+--> /evidence/{id}
+--> /version
|
v
RAG Service
|
+--> Qdrant
+--> BM25 service/index
+--> Reranker
+--> LLM gateway
+--> Redis
  
```

Endpoint مثال

POST /ask

```
{
  "question": "What should be done for suspected acute stroke?",
  "language": "en"
}
```

:Response

```
{
  "status": "supported",
  "answer": "...",
  "citations": [...],
  "latency_ms": 1240,
  "index_version": "who-pen-v1"
}
```

9.3 Redis Caching

ماذا نcache؟

.normalized query → QueryPlan

.normalized query + index version → retrieval result

.repeated public guideline questions → final response إذا policy تسمح.

Cache key

sha256(index_version + normalized_query + pipeline_config_hash)

لا تستخدم query فقط، وإلا ستعيد نتائج من index قديم.

9.4 Vector DB Optimization

في Qdrant production:

.collection versioning

.payload indexes module_id/content_role عند الحاجة.

.HNSW tuning بعد benchmark.

.snapshot backups

.avoid local-file lock model في multi-worker deployment: استخدم Qdrant server.

9.5 CI/CD for RAG

كل commit يغير parsing/chunking/retrieval يجب أن يشغل regression suite.

```

git push
|
v
CI
+--> unit tests
+--> schema tests
+--> no-secret scan
+--> retrieval regression sample
+--> citation validator
+--> numeric guard tests
|
v
Build Docker image
|
v
Staging
|
v
Evaluation gate
|
v
Production / reject

```

9.6 Security

Secrets

لا تخزن API key في:

.notebook output

.run manifest

.GitHub

.Dockerfile

.screenshots

استخدم environment variables أو secret manager.

حادثة فعلية في المشروع

تم إنشاء run manifest باستخدام asdict(CFG) فدخل llm_api_key في JSON. تم اكتشاف المشكلة قبل النشر، والحل الصحيح:

.rotate compromised key

.remove key from manifest

.store only llm_api_key_configured: true/false

add secret scanning في CI.

9.7 Observability

سجل لكل request:

.request_id

.query hash

.planner output

.dense/BM25 top ranks

.reranker latency

.evidence IDs

.answer status

.citation count

.verifier result

.token usage

.total latency

لكن لا تسجل PHI خام بلا policy واضحة إذا تحول النظام إلى patient-specific workflow.

9.8 Production Testing Matrix

قبل launch:
Test
Example
Exact table
AUDIT score range
Numeric threshold
BP/glucose/risk thresholds
Protocol
suspected stroke
Framework
5A's / 5R's
Definition
comorbidity
Cross-module ambiguity
generic WHO PEN vision
Out of scope
question not covered
Adversarial
"ignore WHO PEN and answer from memory"

Citation
 every recommendation maps to evidence
 Empty retrieval
 safe refusal
 API failure
 graceful degradation

9.9 Held-Out Final Evaluation

قبل وصف النظام بأنه production-ready:

- .Freeze architecture
- .Build fresh 30–50 held-out questions
- .Prefer clinician review
- .Do not inspect individual failures أثناء tuning
- .Run retrieval + answer-level metrics once
- .Evaluate numeric and citation gates
- .Sign off version

Deployment Roadmap 9.10 المقترح

Phase 1 — Close Evaluation

- .Solve RAGAS judge quota/dependency path
- .Pilot 5–10
- .Full held-out answer-level evaluation

Phase 2 — Production Hardening

- .secrets cleanup
- .config separation
- .API wrapper
- .structured errors
- .timeouts/retries
- .health checks

Phase 3 — Deploy

- .Docker
- .FastAPI
- .Qdrant server
- .Redis

.staging URL

Phase 4 — Testing

.functional

.retrieval regression

.safety

.load/latency

.failure injection

Phase 5 — Portfolio Packaging

.GitHub repository

.architecture diagram

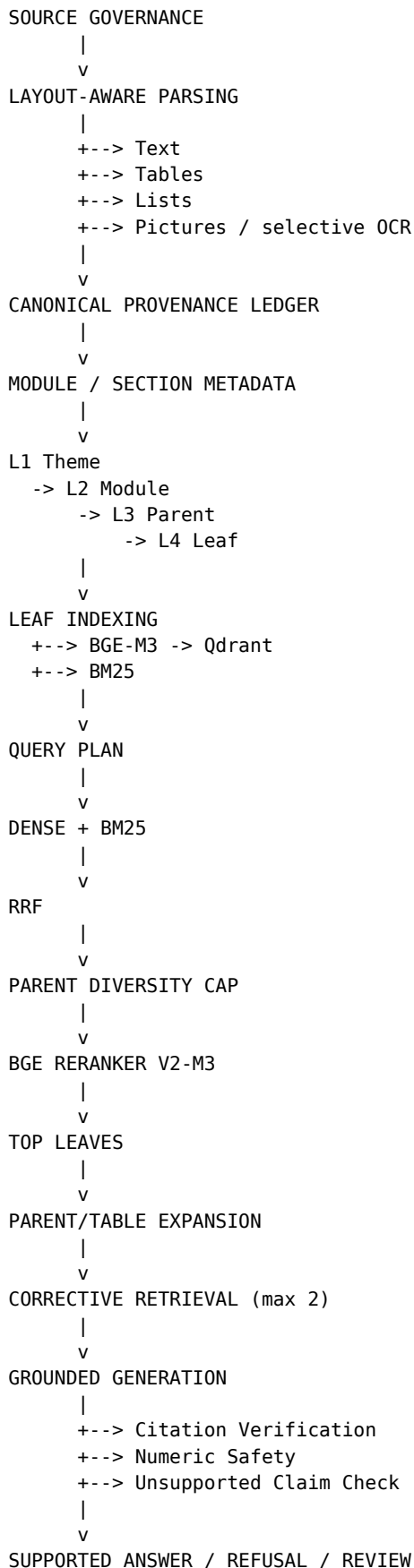
.evaluation report

.demo video

.LinkedIn technical post

10. ملاحق هندسية

Appendix A — المعمارية النهائية المختصرة



Appendix B — Final Retrieval Configuration Snapshot

بدون أي .secrets

```
chunking:
  parent_max_tokens: 760
  parent_overlap_ratio: 0.12
  leaf_max_tokens: 220
  leaf_overlap_ratio: 0.12

models:
  embedding: BAAI/bge-m3
  reranker: BAAI/bge-reranker-v2-m3

retrieval:
  dense_k: 60
  bm25_k: 60
  fusion_k: 50
  rerank_k: 30
  final_leaf_k: 10
  final_parent_k: 5
  max_leaves_per_parent: 3
  rrf_constant: 60
  max_context_tokens: 6000
  corrective_retrieval_loops: 2
  hard_filter_confidence: 0.92
```

Appendix C — Final Run Artifact Structure

```
who_pen_medical_rag_v6_clean/
├── source/
│   └── source_manifest.json
├── cache/
│   └── docling_document.json
├── canonical/
│   ├── module_ranges.parquet
│   ├── page_meta.json
│   ├── elements.parquet
│   └── tables.jsonl
├── chunks/
│   ├── parents.parquet
│   └── leaves.parquet
├── index/
│   ├── leaf_embeddings.npy
│   ├── leaf_embeddings.meta.json
│   ├── bm25.joblib
│   ├── qdrant/
│   └── qdrant.meta.json
├── eval/
├── visual_qa/
└── logs/
    └── run_manifest.json
```

Appendix D — تطور نسخ المشروع

Base
|
v

```
v2
|
v
v3 Timed AutoBatch
|
v
v4 DriveFixed
|
v
v5 NumPyFixed
|
v
v6 Clean
|
v
v6.1 AutoResume
|
v
v6.2 ColabStable
```

ماذا تعكس هذه النسخ؟

تحسين runtime instrumentation

إصلاح Drive/checkpoint behavior

NumPy/package stability

تنظيف module metadata

lossless tables

cache fingerprinting

resume-safe startup

Colab memory stabilization

Appendix E — Root-Cause
Trace Template

استخدم هذا الجدول لأي failure جديد:

Field

Value

question_id

expected module

dense rank

BM25 rank

RRF rank

rank after parent cap

?reranker input

yes/no

final rank

?same parent present

?source anchor present

?metadata filter active

root cause

fix category

query/chunk/index/rerank/gold

Appendix F — Glossary

المصطلح	
المعنى	
RAG	Retrieval-Augmented Generation
Chunk	
جزء من المستند	
Leaf	
أصغر retrieval unit	
Parent	
سياق أكبر مرتبط بال leaf	
Embedding	
vector representation للنص	
Dense Retrieval	
semantic vector search	
Sparse Retrieval	
BM25 مثل lexical term search	
RRF	
Reciprocal Rank Fusion	
Reranker	
نموذج يعيد ترتيب candidates	
Provenance	
أصل evidence: document/page/section	
Grounding	
إلزام answer بالمصدر	
Faithfulness	
مدى دعم context لل claims	
Crag	
Corrective Retrieval-Augmented Generation	
Gold Dataset	

أسئلة مع ground truth للتقييم

Held-Out Set

test set غير مستخدم أثناء tuning

Calibration

تحويل score إلى probability ذات معنى

Citation Binding

ربط claim بمصدر محدد

Appendix G — Deep-Dive Implementation Patterns

G.1 خوارزمية عملية لاكتشاف Headers/Footers المتكررة

بدلاً من الاعتماد على قائمة كلمات ثابتة، يمكن بناء detector إحصائي:

Input

لكل page:

```
[ (text_block, bbox, page_no), ... ]
```

Processing

قسّم الصفحة إلى top band و body و bottom band، مثلاً أعلى/أسفل 8-12% من الارتفاع.

طبع whitespace فقط، ولا تغيّر الأرقام الطبية.

احسب frequency لكل normalized string عبر الصفحات.

اعتبر block noise مرشحاً إذا اجتمعت:

position ثابتة تقريباً.

frequency عالية.

length قصيرة/متوسطة.

لا يبدو heading حقيقياً مرتبطاً بمحتوى الصفحة.

احتفظ audit trail: ما الذي حذف؟ من أي صفحات؟

Pseudocode

```
from collections import Counter

candidate_counter = Counter()

for page in pages:
    for block in page.blocks:
        if in_top_or_bottom_band(block.bbox, page.height):
            key = light_normalize(block.text)
            candidate_counter[key] += 1

repeated = {
    text for text, freq in candidate_counter.items()
    if freq >= 0.35 * total_pages
}
```

Output

```
clean_blocks
removed_noise_ledger
```

لماذا نحتاج ledger؟

لأن أي preprocessing في المجال الطبي يجب أن يكون reversible/auditable. إذا اختفى heading مهم، تستطيع معرفة هل تم حذفه كheader بالخطأ.

Graph Ordering كمسألة G.2 Reading Order

يمكن النظر إلى page blocks كnodes في graph.

لكل block:

node = {x0, y0, x1, y1, type, text}

ننشئ علاقات:

above(A,B)

same_column(A,B)

left_of(A,B)

heading_of(H,P)

ثم نرتب بالقيود بدلاً من y فقط.

Input → Processing → Output

```

Bounding boxes
|
v
Column clustering
|
v
In-column topological ordering
|
v
Heading/list/table attachment
|
v
Semantic reading sequence

```

Cross-column test

أنشئ regression page تحتوي عمودين وتأكد أن sequence المتوقع:

A1 A2 A3 B1 B2 B3

وليس:

A1 B1 A2 B2 A3 B3

G.3 Table Canonicalization Pipeline

Input

Detected table grid + merged cells + caption + page

Processing

1. Recover rows/columns
2. Expand/track merged cells
3. Preserve original headers
4. Create unique internal column IDs
5. Normalize cell whitespace safely
6. Serialize row semantics
7. Create retrieval_text
8. Keep Markdown + raw schema + provenance

Retrieval text template

بدلاً من embedding الجدول كـ Markdown فقط، يمكن إنشاء نص row-aware:

Module: 2.7
 Table: Expected relative risks ...
 Row:
 Disease = COPD
 RR women = 2.3 (1.7–3.1)
 RR men = 1.9 (1.2–3.1)

هذا يجعل كل row self-describing، ويقلل اعتماد retrieval على موضع العمود.

Output

```
{
  "table_id": "table_...",
  "page": 213,
  "caption": "...",
  "columns": [...],
  "rows": [...],
  "markdown": "...",
  "retrieval_text": "..."
}
```

G.4 Selective OCR Triage

بدلاً من OCR لكل صورة، استخدم triage score.

[OCRPriority = w_1 ImageArea + w_2 TextScarcity + w_3 SlideLikelihood + w_4 QAFlag]

حيث:

ImageArea: نسبة مساحة الصورة من الصفحة.

TextScarcity: قلة native text.

SlideLikelihood: وجود rectangular slide layout.

QAFlag: failure في manual/automated audit.

إذا تعدى score threshold، نفذ OCR فقط لهذه الصفحة/region.

Use Case

صفحة بها slide تعرض 5R's في صورة، ويتبعها facilitator note كنص native. يجب ربط OCR text مع note تحت نفس module/section دون دمجها في block واحد غير قابل للتتبع.

G.5 مقارنة أدوات

Parsing التي تمت

دراساتها

الأداة

نقاط القوة

القيود

دورها الأنسب في هذا المشروع

PyMuPDF

page rendering, raw text, coordinates, سريع

لا يعيد بناء semantic tables بمفرده

audit, sanity checks, visual QA

Docling

layout items, tables, provenance, structured document

أثقل حسابياً وذاكرياً

parser الرئيسي

LlamaParse

parsing متقدم كخدمة، جيد للمستندات المعقدة

external dependency, cost/privacy/latency considerations

core dependency وليس fallback/benchmark

OCR engine

ينقذ scanned/embedded text

cost و noise

selective only

القرار المعماري لم يكن اختيار "أفضل parser مطلقاً"، بل بناء stack يمكن أن يستفيد من أكثر من أداة حسب failure mode.

G.6 بناء Hierarchy خطوة بخطوة

Input

Canonical elements مرتبة مع:

page, module, section, content_role, text, table_id

Processing

Step 1 — L1/L2

استنتاج/تثبيت Theme و Module من page ranges الصحيحة.

Step 2 — L3 Parent Construction

نجمع العناصر المتجاورة بشرط:

نفس module.

section متوافق.

لا نتجاوز parent token budget.

لا نكسر table row group بشكل اعتباطي.

Step 3 — L4 Leaf Construction

نقسم parent إلى leaves صغيرة، مع 12% overlap، ونحافظ على parent_id.

Pseudocode

```
for module in modules:
    for section in module.sections:
        parents = pack(section.elements, max_tokens=760)
        for parent in parents:
            leaves = sliding_semantic_pack(
                parent.elements,
                max_tokens=220,
                overlap=0.12,
            )
            for leaf in leaves:
                leaf.parent_id = parent.id
```

Output

parents.parquet
leaves.parquet

Invariants يجب اختبارها

كل leaf لها parent موجود.

parent range leaf لال page range

parent لال leaf = module_id لال module_id

orphan chunks لا توجد

table_id تحتفظ table chunks

source hash واحد ومتسق.

G.7 Module Range Regression Checks

unit tests يحتاج Annex, module mapping بعد مشكلة

أمثلة ranges المعتمدة في الإصلاح:

Module
PDF pages
Front matter
26-1
1.1
50-27
1.2
72-51
2.1
92-73
2.2
120-93
2.3
148-121
2.4
170-149
2.5
188-171
2.6
204-189
2.7
222-205
3.1
244-223
3.2
274-245
3.3
306-275

3.4
330-307
3.5
360-331
3.6
392-361
3.7
418-393
3.8
442-419
3.9
474-443
4.1
504-475
4.2
558-505
5.1
600-559
5.2
628-601
Annex 1
644-629

Regression invariant:

```
union(all module ranges) == pages 1..644
intersection(any two ranges) == empty
```

مع السماح بأن بعض الصفحات blank من ناحية provenance.

G.8 Dense Retrieval — Input / Processing / Output

Input

```
semantic_query = "cardiovascular risk chart ten year risk"
```

Processing

- .tokenize query according to BGE-M3 tokenizer
- produce 1024-dimensional embedding في إعدادات المشروع.
- .query Qdrant ANN index
- .optionally apply safe metadata filtering

.return top 60 leaves

Output

```
[
  {"leaf_id": "leaf_A", "dense_score": 0.72, "rank": 1},
  {"leaf_id": "leaf_B", "dense_score": 0.70, "rank": 2}
]
```

Failure mode

Semantic match قد يجلب "risk assessment" من module قريب لكنه ليس الجدول المطلوب. هنا يأتي BM25 + .reranker

G.9 BM25 — Input / Processing / Output بالتفصيل

Input

lexical_query = "AUDIT 16 19 alcohol intervention"

Processing

.tokenize query

IDF لل terms النادرة.

term frequency داخل كل leaf.

.length normalization

Output

.Top 60 sparse hits

Failure mode

إذا السؤال paraphrase لا يحتوي المصطلح نفسه، BM25 قد يفشل بينما dense ينجح.

G.10 RRF — Worked Example

لدينا أربع وثائق:

Doc
Dense rank
BM25 rank
A
1
8
B
2
2
C
5
1
D
3
—

باستخدام k=60:

- [RRF(A)=1/61+1/68]
- [RRF(B)=1/62+1/62]
- [RRF(C)=1/65+1/61]
- [RRF(D)=1/63]

غالباً B يصعد لأنه قوي في القنوات. هذه خاصية جيدة في guideline retrieval: الاتفاق بين semantic وlexical signals يزيـد الثقة النسبية.

G.11 Cross-Encoder — Input / Processing / Output

Input

query + 30 fused candidate leaves

Processing

[CLS] query [SEP] document [SEP]

لكل candidate: يمر الاثنان معاً عبر transformer ويخرج relevance logit.

Output

ranked 10 leaves + raw reranker logits

Failure modes التي ظهرت

Reranker Demotion: evidence كان rank ممتاز قبل reranking ثم نزل.

Rerank Input Cutoff: evidence الصحيح كان $\text{fused rank} > 30$, فلم يره reranker أصلاً.

لماذا لم نرفع rerank_k عشوائياً؟

لأن رفعه إلى 50 كجزء من V2 لم يحسن coverage بشكل آمن على كل الأسئلة، وزاد interaction مع scoring الآخر.

G.12 Parent Expansion — Input / Processing / Output

Input

:Final leaf hits

```
leaf_1 -> parent_A
leaf_2 -> parent_A
leaf_3 -> parent_B
```

Processing

.group by parent_id

parent score = max child reranker score في التنفيذ الحالي.

.sort parents

skip roles غير مسموحة للgeneration.

.enforce max_context_tokens

.keep top 5 parents

Output

:Evidence Pack

```
[
  {
    "evidence_id": "E1",
    "parent_id": "parent_A",
    "module_id": "3.4",
    "page_start": 312,
    "text": "...",
  }
]
```

G.13 Query Planner — أمثلة آمنة وخطرة

Safe routing

:Question

What is the main CVD assessment skill in module 3.1?

إذا النص يذكر module صراحة يمكن confidence عالية.

Risky routing

:Question

What is the vision of WHO PEN?

هذا broad question. إجباره على module محدد قد يخفي front matter أو overview.

المقترحة Production rule

module_id hard filter:

allowed only when explicit/very high confidence

content_role hard filter:

do NOT activate alone

prefer soft boost or post-filter

Appendix H — End-to-End WHO PEN Case Studies

H.1 Case Study: AUDIT Alcohol Screening

السؤال

What intervention is associated with an AUDIT score of 16–19?

لماذا صعب؟

.numeric range

.table semantics

.exact row required

Pipeline

Query

```
|
+--> semantic: alcohol AUDIT intervention
+--> lexical: AUDIT 16 19
|
```

Dense + BM25

```
|
```

RRF

```
|
```

Table-aware leaf ranks high

```
|
```

Reranker confirms row relevance

```
|
```

Parent/table context expansion

```
|
```

Grounded answer + citation

```
|
```

Numeric guard validates 16–19 exactly

أهم نقطة

لو parser فقد row/header relationship, كل المراحل التالية تصبح غير موثوقة.

H.2 Case Study: Smoking Cessation — 5A's / 5R's

سؤال محتمل

What are the steps of the 5A's approach for tobacco cessation?

تحديات

.acronym-like ordered framework

.list order مهم.

قد يظهر في slide + facilitator notes.

Retrieval strategy

BM25 قوي لـ 5A والـ terms.

Dense يفهم cessation counselling.

Parent expansion يحافظ على ordered steps.

Verification

لا يسمح generator بإضافة step سادسة من external memory.

H.3 Case Study: CVD Risk Chart

سؤال

What does the WHO PEN CVD risk chart provide?

تحدي

قد توجد عدة صفحات فيها كلمات .risk, CVD, assessment.

Pipeline

Dense يجلب BM25 conceptual relevance, يجلب reranker exact chart phrase, يفرق بين learning objective والـ definition, و citation يعيد page الصحيحة.

Failure class

Reranker demotion ظهرت في أحد أسئلة CVD, ما يثبت أن reranker ليس دائماً monotonic improvement.

H.4 Case Study: Household Air Pollution / Biomass

السؤال

What household fuel exposure is highlighted as a major source of harmful household air pollution?

ما حدث فعلياً

Gold exact anchor لم يصل، لكن evidence parents احتوت:

.cooking with biomass fuel

.poor household air quality

.cleaner fuels

.exposure to PM2.5 due to biomass cooking

evaluation درس

Exact string anchor = false negative أحياناً إذا كان semantic evidence كافياً.

H.5 Case Study: Suspected Acute Stroke

السؤال

How does WHO PEN characterize suspected acute stroke?

الفعلي Evidence pack

Parent من 3.4 module تضمن:

.sudden neurological deficit

.FAS

.time of onset

.suspicion of acute stroke

.emergency services

.immediate transfer

لكن Stroke is a medical emergency exact anchor لم يكن في evidence pack.

القرار

لا نعيد تصميم retriever بسبب exact-anchor miss إذا manual clinical adjudication يؤكد وجود الإجراء المطلوب، لكن يجب تحسين evaluation labels مستقبلاً.

Appendix I — Evaluation Design in Greater Depth

I.1 Stratified Gold Sampling

لا تختَر 100 سؤال عشوائي فقط. كَوْن strata:

Stratum

الهدف

definitions

semantic retrieval

table numeric

table preservation + lexical

threshold

numeric safety

protocol steps

ordered context

emergency

high safety impact

risk factors

list completeness

learning objective/scope

metadata discrimination

cross-module ambiguity

routing robustness

ثم تأكد أن كل module ممثل.

Clinical RAG I.2 Precision vs Recall

لو رفعنا recall بجلب 50 parent إلى generator، قد تزيد فرصة وجود evidence الصحيح، لكن يزيد noise و prompt length، وقد يسبب hallucination by distraction.

لو خفضنا إلى parent واحد، precision ترتفع لكن قد نفقد required evidence.

لذلك design الهدف:

```
High recall in candidate stage
|
v
Aggressive reranking
|
v
Moderate evidence pack
```

I.3 Bootstrap Confidence Intervals

لأن 100 سؤال ليست population كاملة، يمكن حساب uncertainty عبر bootstrap:

```
scores = []
for _ in range(5000):
    sample = df.sample(len(df), replace=True)
    scores.append(sample.hit_at_10.mean())

ci = np.quantile(scores, [0.025, 0.975])
```

هذا مفيد عند مقارنة 0.95 0.97 vs؛ الفرق قد لا يكون meaningful إحصائياً.

I.4 Failure Review Protocol

لكل miss:

افتح source page.

اقرأ gold anchor.

افحص dense top-200.

افحص BM25 top-200.

افحص RRF rank.

افحص parent cap.

افحص rerank input.

افحص final top-k.

افحص same parent.

افحص semantic equivalent evidence.

قرر: model failure أم label failure؟

لا تعدل hyperparameters قبل هذه السلسلة.

I.5 لماذا Gold-100 لم تعد Held-Out؟

كل مرة تنظر إلى failures ثم تغير النظام بناءً عليها، أنت تستخدمها كdevelopment data.

```

Gold-100
|
Failure inspection
|
Parameter change
|
Re-evaluate Gold-100
  
```

هذه loop = tuning

الحل النهائي:

```

Development Gold-100 -> tune/freeze
Fresh Held-Out 30-50 -> one-shot final evaluation
  
```

Appendix J — Production Engineering Blueprint

J.1 Repository Structure

```

WHO-PEN-Clinical-RAG/
├── app/
│   ├── main.py
│   ├── api/
│   │   ├── routes.py
│   │   └── schemas.py
│   └── rag/
│       ├── planner.py
│       ├── dense.py
│       ├── sparse.py
│       ├── fusion.py
│       ├── reranker.py
│       ├── evidence.py
│       ├── generation.py
│       └── verification.py
├── ingestion/
│   ├── source_governance.py
│   ├── parsing.py
│   ├── tables.py
│   ├── metadata.py
│   └── chunking.py
├── evaluation/
│   ├── retrieval_eval.py
│   ├── evidence_eval.py
│   ├── ragas_eval.py
│   ├── safety_eval.py
│   └── datasets/
├── configs/
│   └── settings.py
├── tests/
│   ├── test_metadata.py
│   ├── test_tables.py
│   ├── test_retrieval.py
│   ├── test_citations.py
│   ├── test_numeric_safety.py
│   └── test_api.py
├── docs/
│   ├── architecture.md
│   ├── evaluation.md
│   └── troubleshooting.md
├── notebooks/
│   └── WHO_PEN_Production_Clinical_RAG_v6_2.ipynb
├── Dockerfile
├── docker-compose.yml
├── requirements.lock
├── .env.example
├── .gitignore
└── README.md

```

J.2 API Layer Separation

لا تجعل route نفسها تحتوي retrieval logic.

```

@app.post('/ask')
async def ask(req: AskRequest):
    return await rag_service.answer(req)

```

ثم rag_service ينفذ pipeline.

الفائدة:

.unit testing
 .swapping LLM provider
 .background evaluation
 .tracing

J.3 Timeout Budget

مثال SLO budget مبدئي:

Stage	Budget
query planning	1.5s-0.5
dense + BM25	300ms محلياً غالباً
reranker	1.5s-0.3 حسب hardware
generation	5s-1
verifier	4s-1

ليس المقصود أن هذه أرقام مضمونة؛ يجب benchmark فعلي على deployment hardware. المهم وضع budget لكل stage حتى تعرف أين latency تتسرب.

J.4 Retry Policy

لا تستخدم retry نفسه لكل الأخطاء.

429 -> exponential backoff / Retry-After
 5xx -> limited retry
 4xx auth -> no retry
 validation error -> no retry
 empty retrieval -> corrective path, not transport retry

LLM Provider J.5 Circuit Breaker

إذا provider يعيد 429/5xx باستمرار:

closed -> failures exceed threshold -> open
 open -> fail fast / fallback
 cooldown -> half-open probe
 success -> closed

هذا يمنع request pile-up.

J.6 RAGAS في CI؟

لا تشغيل full RAGAS على كل commit إذا كان مكلفاً وبطيئاً.

مقترح:

10–20 PR: retrieval regression سؤال deterministic.

nightly: larger retrieval + safety

release candidate: full answer-level evaluation

J.7 Secret Scanning

أضف pre-commit/CI checks لأي patterns مثل:

```
sk-
GROQ_KEY_PREFIX
OPENAI_API_KEY=
LLM_API_KEY=
```

مع منع commit إذا وجد secret محتمل.

J.8 Index Versioning

Version identifier يمكن أن يكون hash من:

```
source_sha256
+ parser_version
+ chunk_config
+ embedding_model
+ metadata_schema_version
```

إذا تغير أي منها، لا تستخدم index القديم بصمت.

J.9 Blue/Green Index Deployment

Current API -> index_v1
Build index_v2 offline
Evaluate index_v2
Deploy index_v2 alongside v1
Switch traffic
Monitor
Rollback to v1 if needed

هذا أكثر أماناً من overwrite collection في مكانها.

J.10 Final Go-Live Checklist

Data

.source hash verified
.page count verified
.canonical provenance QA passed
.tables audited

Retrieval

.held-out Hit@K acceptable
.failure cases adjudicated
.no unsafe hard filtering

Generation

.grounded prompt
.citation binding
.numeric verification
.refusal path

Evaluation

.RAGAS/answer-level complete
.critical numeric mismatch = 0
.invalid citation = 0
.unsupported clinical claims = 0 on acceptance set

Security

.keys rotated
.no secrets in Git history
.env.example only placeholders.

Operations

.health checks

.logging/tracing

.latency dashboard

.rollback

.backup/snapshot

Appendix K — Chronological Engineering Journey

K.1 من Prototype إلى v6.2

Build the full pipeline : المرحلة الأولى:

تم بناء end-to-end architecture تغطي source governance حتى RAGAS.

Runtime instrumentation : المرحلة الثانية:

ظهرت الحاجة لمعرفة أي cells تستغرق الزمن فعلياً، فتم إدخال stage timing و auto batch sizing.

Drive / checkpoint reliability : المرحلة الثالثة:

بسبب Colab lifecycle، أصبح checkpoint جزءاً من architecture، وليس convenience فقط.

NumPy / dependency stability : المرحلة الرابعة:

ظهرت runtime/package interactions، فتم إضافة preflight checks وتثبيت مسار أكثر استقراراً.

Metadata & table correctness : المرحلة الخامسة:

أهم مرحلة من ناحية جودة retrieval:

إصلاح Annex 1.

منع module overlap.

duplicate-header table protection.

table-aware retrieval text.

stale-index fingerprinting.

AutoResume / ColabStable : المرحلة السادسة:

reuse parsed cache عند صلاحيته.

avoid unnecessary DocumentConverter creation عند cache hit.

move expensive parsing path إلى CPU عند الحاجة للاستقرار.

reserve GPU لل embedding/reranking.

Evaluation-driven refinement : المرحلة السابعة:

.Gold-100

.root-cause traces

.gold repair

.V2 ablation
 .coverage-first selection
 .evidence-pack evaluation

Answer-level evaluation blocker: المرحلة الثامنة:

.RAGAS dependency mismatch
 .fix compatibility
 .Groq 429 even for one-question probe
 decision: لا نزور production gate; نسجل RAGAS pending ونغلقها عند توفر judge quota/provider مناسب.

K.2 ما الذي لم نفعله عمداً؟

هذا مهم في أي مقابلة تقنية؛ المعماري الجيد يعرف ما الذي لم يضيفه ولماذا.
 لم نستخدم GraphRAG بلا حاجة مثبتة.
 لم نستخدم RAPTOR بدلاً من source hierarchy.
 لم نعمل global OCR لكل 644 صفحة بسبب resource/noise trade-off.
 لم نقبل Full V2 لأن MRR تحسن بينما coverage انخفض.
 لم نصف reranker sigmoid بأنه probability calibrated.
 لم نعتبر Gold-100 final held-out بعد استخدامها في tuning.
 لم نعلن Production Passed مع RAGAS غائبة.
 هذه القرارات أهم من قائمة technologies؛ لأنها تعكس engineering discipline.

الخلاصة المعمارية

بناء Production-Grade Medical RAG ليس مشروع "اختيار LLM + Vector DB". النجاح الفعلي يعتمد على سلسلة كاملة:

- Source Integrity
 - Layout Integrity
 - Table Integrity
 - Metadata Integrity
 - Chunk Integrity
 - Retrieval Coverage
 - Reranking Quality
 - Context Expansion
 - Grounded Generation
 - Claim Verification
 - Numeric Safety
 - Evaluation Integrity
 - Production Observability

في هذا المشروع، الأخطاء الأكثر قيمة تعليمياً لم تكن "النموذج ضعيف"، بل:

.module metadata contamination

.table header loss

.stale index reuse

.reranker cutoff/demotion

.incomplete gold labels

.hard-filter risk

.dependency instability

.API rate limits

.secret serialization

وهذا هو جوهر هندسة RAG الحقيقية: تشخيص النظام كمنظومة متعددة الطبقات، وليس محاولة إصلاح كل مشكلة بتغيير LLM أو زيادة Top-K عشوائياً.

النسخة الحالية وصلت إلى retrieval/evidence performance قوية على development suite، لكن المسار المهني الصحيح قبل إعلان Production هو إغلاق answer-level evaluation على held-out set، ثم نشر نسخة API قابلة للرصد والاختبار مع security و CI/CD و rollback واضح.

مراجع المشروع الداخلية المستخدمة في إعداد هذه الوثيقة

WHO_PEN_Production_Clinical_RAG_v6_2_ColabStable.ipynb

WHO_PEN_Production_Clinical_RAG_v6_Clean.ipynb

WHO_PEN_Production_Clinical_RAG_v6_1_AutoResume.ipynb

WHO_PEN_Production_Clinical_RAG_v5_NumPyFixed.ipynb

.Gold-100 retrieval failure-analysis outputs

.Gold-100 evidence-pack evaluation output

.Final run manifest and production gate definitions

Security note: لا تحتوي هذه الوثيقة على أي API keys أو secrets. أي مفتاح ظهر في logs أو manifest سابق يجب اعتباره compromised ويتم تدويره قبل GitHub أو deployment.